Shiv Nadar University - Chennai

Department of Computer Science

25CS2741 Fundamentals of Internet of Things + Lab

Date: 17/08/2026 (Week 5) **Continuous Lab Evaluation-6** II Yr/ 3rd Sem B. Tech CSE(IoT)A&B

Allotted Marks: 10 (Hardware Implementation + Code Efficiency + Viva-Voce)
Topics: 8051 Tool Chain setup – Development Board – AT89S52
Strictly No to using **Arduino APIs, HAL libraries, RTOS services!**
Supporting Documents: **1.** Board Schematics 2. Pin diagram of 8051
Hardware Debug: Multi meter.

UART Driver and Serial Debugging

Problem Statement:

Develop a basic interrupt-driven UART driver for the 8051 and use a PC terminal such as PuTTY to transmit and receive characters.

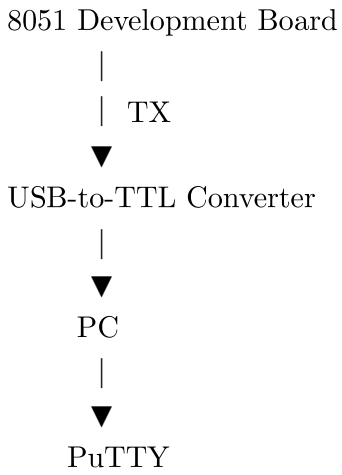
Develop an interrupt-driven **UART driver for the 8051 that communicates with a PC terminal through a USB-to-TTL serial interface**. Configure the UART for **9600-8-N-1** communication and demonstrate character transmission and reception using PuTTY.
Implement UART transmit and receive functions, followed by an interrupt-driven RX ring buffer.

Task A — UART Initialization and Character Transmission

Configure the 8051 UART for:

- 8-bit UART mode
- Asynchronous operation
- 9600 baud
- 1 stop bit
- No parity

Transmit:
Hello from 8051
continuously to a PC terminal.



Task B — UART Echo

Now make the experiment interactive.

Requirement

Whatever character is typed in PuTTY must be returned by 8051.
Example:
If the student types:
ABC123
the terminal should display:
ABC123

Follow these hints to understand and develop the driver :

Stage 1

Hint 1 : Which two pins of the AT89S52 are associated with serial communication?

Hint 2: Look at the AT89S52 pin description. Are TXD and RXD ordinary GPIO pins, or do they have an alternate peripheral function?

Stage 2 — Discover the UART Configuration Register

Hint 3: The AT89S52 serial port supports multiple operating modes. Find the register that selects the serial operating mode.

Hint 4: Which bits of that register determine the serial mode?

Hint 5: What is baud rate? Which UART mode provides 8-bit asynchronous communication with a variable baud rate?

Hint 6: What value must be placed in that serial configuration register to select the chosen mode above and enable reception?

Stage 3 — Discover the Transmit Register

Hint 7: When the 8051 wants to transmit a character, where does the software place that character for transmission ?

Hint 8: Does the above register only used for transmission?

Stage 4 — Discover How Transmission Completes

- After writing a character to serial register, how does the software know that the character has finished transmitting? Any flag indication!?
- What happens to that flag when transmission is complete?
- Draw the driver sequence for the transmission ?

Stage 5 — Discover Baud Rate Generation

1. UART communication requires both sides to agree on a data rate. Which 8051 peripherals normally provides the baud-rate timing?
2. Which mode is commonly used for UART baud-rate generation? Why is it useful for UART?

Stage 6 — Derive 9600 Baud

Now give them the crystal frequency of your board.

For example: 11.0592 MHz

Find the 8051 Timer 1 baud-rate equation in the datasheet. What timer reload value is required for 9600 baud?

Stage 7 — First Programming Challenge

Using the information discovered in Stages 1–6, write a function uart_init() that configures the 8051 UART for 9600-8-N-1 communication.

Stage 8 — First Verification

How can you verify that your UART configuration is working without writing a complete communication application?

Transmit one known character.

Stage 9 — Receive a Character

Now reverse the direction.

8051 has received a character from the PC. How does the software know that a character has arrived?

- Investigate SCON and discover:

Which register contains the received character? How to declare the variable to receive the character.

Stage 10 — PuTTY Echo Experiment

Now the question becomes:

Develop an application that continuously receives one character from PuTTY and immediately transmits the same character back.

Expected:

PC → 8051 → PC

Then test: *Hello 8051*

Expected: ***Hello 8051***